BioHackrXiv

BioHackathon series:
EuroBioC 2026 Hackathon, Turku, Finland, 2026
Turku, Finland
Project 2

Submitted: 19 Jun 2026

Published by BioHackrXiv.org

BiocExecute: Make package functions or workflows executable from the command line

Guillaume Deflandre [1], **Leopold Guyot** [1], **Rasmus Hindström** [2], **and Claire Rioualen** [3]

1 Computational Biology and Bioinformatics, de Duve Institute, UCLouvain **2** Department of Computing, University of Turku, Turku, Finland **3** IFB-core, French Institute of Bioinformatics (IFB), CNRS, INSERM, INRAE, CEA, 94800 Villejuif, France

Introduction

Bioconductor (Huber et al., 2015) is a collection of more than 2,400 open-source software packages, together accounting for about a million downloads per year (Bioconductor, 2026). The packages are thoroughly maintained and documented, and their quality is enforced through BiocCheck. Their reach, however, largely stops at the R console.

The R and Bioconductor paradigm of interactivity through a responsive, informative REPL has served academic users well for a long time. But computational biology has grown more interdisciplinary, and increasingly runs on high-performance and high-throughput compute, large-scale experimentation, and the cloud. In these settings analyses are assembled from command-line tools and run under workflow managers, where an interactive R session does not fit. For Bioconductor and the work built on it to stay relevant, its tooling must be portable and scriptable as well as interactive.

Some of this ground is already covered: R2G2 integrates R with Galaxy, and Rapp (r-lib, 2024) lets an R script run as if it were a command-line program. What is missing is a path from a Bioconductor package to such tools that follows the project's own packaging conventions. BiocExecute fills that gap. It is a package that wraps Rapp so that the functions and workflows inside any Bioconductor package can be called from the command line, with these command-line entry points declared and bundled as part of the package itself.

Both users and developers gain from this. Users can run Bioconductor tools outside of R scripts, combine them as modules with other command-line tools, and reuse them in workflows under any workflow manager; developers reach a wider range of users. More broadly, making Bioconductor packages executable improves their FAIRness (Barker et al., 2022; Wilkinson et al., 2016) and gives Bioconductor software visibility among a larger community of bioinformaticians.

Longer term, the goal is to lay the groundwork for programmatic generation of command-line tooling from within Bioconductor packages, so that this tooling can be slotted into modern workflow management systems or interactive platforms such as Galaxy (Goecks et al., 2010).

Implementation

BiocExecute

`BiocExecute` is a package to make Bioconductor/R package functions executable in the Command Line Interface (CLI) (Figure 1).

Figure 1: Hex sticker for the BiocExecute package.

The package comes with a few functions that need to be called upon building a package or upon using the package in the CLI.

How to use executables

Here's a quick example of how you would call the tool `name(x, y)` from the package `pkgExample` as a package *user*:

First, the user needs to make sure the package executables are available:

```
BiocExecute::execInstall("pkgExample")
```

And now call those functions from within the terminal:

```
pkgExample --help
pkgExample name -x Bilbo -y Baggins
```

How to create executables

`BiocExecute` uses `Rapp` to make your package functions/scripts executable. In the coming sections we will show the different ways `Rapp` includes arguments to be called in the CLI. Note that `Rapp` by itself works with scripts in which the first line is defined as `#!/usr/bin/env Rapp`. You should not include this line in your scripts; it is handled internally, bundled in the `BiocExecute` package.

The executables can range widely, from a simple function call to an entire complex workflow. Note that bigger workflows mean more parameters to call in the CLI.

As a maintainer, you may choose which functions/workflows you decide to include in your package. As a package user, we suggest creating pull requests on the package GitHub repository for workflows that you believe should be easily accessible. Try to avoid creating strongly personal workflows. Prioritize workflows that are reused across different use cases.

Create a script

An *R* package that has executables should include them in its `exec/scripts/` directory. `BiocExecute` has all the necessary functions to create these folders, scripts and more.

For instance, suppose the name of my package is `mypkg`. In its root directory, I have neither an `exec` nor a `scripts` folder:

```
mypkg
|   README.md
|   DESCRIPTION
|   NEWS.md
|   NAMESPACE
|
+---R
|   |   functionA.R
|   |   functionB.R
|
+---tests
|   |   testA.R
|   |   testB.R
|
+---vignettes
    |   myVignette.Rmd
```

To create the necessary files, I use:

```
## From the root directory
execSkeleton()
```

Now, my directory tree looks like this:

```
mypkg
|   README.md
|   DESCRIPTION
|   NEWS.md
|   NAMESPACE
|
+---R
|   |   functionA.R
|   |   functionB.R
|
+---exec
|   |   mypkg.R
|   |
|   +---scripts
|       |   base_template.R
|
+---tests
|   |   testA.R
|   |   testB.R
|
+---vignettes
    |   myVignette.Rmd
```

For now, there is a simple template in the `scripts` folder. The same template can be created using `execTemplate()`.

The maintainer of a package should then create the scripts with functions or workflows that

they wish to make executable on the CLI.

Once the `exec/scripts/` directory is created, you should (in order from the root directory):

1. Call `execCompile()` to build the actual executable script. This file will be called by its package name and it will be written in the `exec` folder. Do not edit this file by hand, as it is likely to be overwritten.
2. Call `devtools::install()` to re-install the package with the executables.
3. Call `execInstall("mypkg")` or a vector of packages to make the executables available in the CLI. To remove those, call `execUninstall("mypkg")`. To make the executables available system-wide (with sudo access), use the `destdir` parameter.

Your package functions/tools are now available in the CLI!

Script files

The files in the `exec/scripts/` directory are at the core of the available executables. One script corresponds to one command, which can be a simple function or even a whole workflow. These scripts are combined into the main executable script in `exec/` (see section above).

The script files need to follow the `Rapp` architecture. A template file is built by default to get you started. In practice, these *R* scripts are really a repetition of the important functions within your package, except that their parameters and documentation are re-written in a way for `Rapp` to parse them.

Rapp fields and function parameters

`Rapp` parses *R* scripts by looking for specific expression patterns at the top level. Each pattern maps to a different CLI surface. The table below summarises the most common ones:

R expression	CLI surface
`foo <- ""`	Option: `app --foo value`
`foo <- NULL`	Positional argument: `app foo-value`
`foo <- TRUE`	Boolean switch: `app --foo=true` / `app --foo=true`
`foo <- c()`	Repeatable option (raw strings): `app --foo a --foo b`
`foo <- list()`	Repeatable option (parsed values): `app --foo 1 --foo 2`
`switch("", cmd1 = {}, cmd2 = {})`	Subcommands: `app cmd1 --help`

Annotations are written as YAML hash-pipe comments (`#|`) directly above the assignment they document. The most commonly used fields are:

Field	Description
`description`	Short description shown in `--help` output
`title`	Title for subcommands
`short`	Single-letter alias (e.g. `short: n` enables `-n`)
`required`	Set to `false` to make a positional argument optional
`val_type`	Expected type: `string`, `integer`, `float`, `bool`, or `any`

A minimal example script illustrates how these come together:

```
#| description: Count word occurrences in a file.

#| description: Path to the input file.
inputFile <- NULL
```

```
#| description: Word to count.
#| short: w
word <- ""

#| description: Print each match.
verbose <- FALSE
```

Called from the terminal:

```
count-words myfile.txt --word hello --verbose
count-words --help
```

Use `execTemplate()` to create a template script with the different parameters to facilitate writing your scripts.

For a full description of all available fields and advanced patterns such as nested subcommands, refer to the Rapp GitHub page.

Conclusion

BiocExecute exposes the functions and workflows of a Bioconductor package as command-line tools, bundled and installed as part of the package itself. A maintainer writes the commands as Rapp scripts under `exec/scripts/`, combines them into a single entry point, and installs it; users then call the tools directly from the terminal and drop them into any workflow. This brings interactive Bioconductor tools into the scriptable, command-line environment that high-throughput and cloud analyses are built on, without asking developers to maintain a separate codebase.

Several design questions remain open. Where app-ification should happen, whether at package build time, at installation, or on demand, is not yet settled. Neither is how much validation, such as package-version and argument-type checking, the executables should enforce by default, given the trade-off between safety and overhead. A third question is how these tools should fail gracefully, with informative messages rather than R tracebacks. Answering them is the next step toward programmatic generation of command-line tooling across Bioconductor.

Data availability

All scripts and materials developed during the hackathon are available in the BiocExecute GitHub repository.

Acknowledgements

This work received state aid managed by the National Research Agency under France 2030 for the French Institute of Bioinformatics (IFB), funded by the Investments for the Future Program, number ANR-11-INBS-0013, as well as for structural research equipment / EQUIPEX+ with reference ANR-21-ESRE-0048.

Barker, M., Chue Hong, N. P., Katz, D. S.others. (2022). Introducing the FAIR Principles for research software. *Scientific Data*, *9*(1), 622. https://doi.org/10.1038/s41597-022-01710-x

Bioconductor. (2026). *Download statistics for Bioconductor software packages*. https://bioconductor.org/packages/stats/.

Goecks, J., Nekrutenko, A., Taylor, J., & The Galaxy Team. (2010). Galaxy: A comprehensive approach for supporting accessible, reproducible, and transparent computational research in the life sciences. *Genome Biology*, *11*(8), R86. https://doi.org/10.1186/gb-2010-11-8-r86

Huber, W., Carey, V. J., Gentleman, R.others. (2015). Orchestrating high-throughput genomic analysis with Bioconductor. *Nature Methods*, *12*(2), 115–121. https://doi.org/10.1038/nmeth.3252

r-lib. (2024). *Rapp: Command line applications with R*. https://github.com/r-lib/Rapp.

Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J.others. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, *3*, 160018. https://doi.org/10.1038/sdata.2016.18

Author contributions

Guillaume Deflandre: Hacking, Writing Leopold Guyot: Hacking, Writing Rasmus Hindström: Hacking, Writing Claire Rioualen: Hacking, Writing